

Deep Learning & GenAI Project Report

Name: Kamal Kalawatia
Roll No.: 24f1000134

1. Abstract / Executive Summary

This project tackles the Smart MCQ Solver Kaggle-style competition, which frames multiple-choice question answering as a **per-option relevance-scoring / ranking problem** rather than a standard single-label classification task. Each question offers five options (A–E), and the objective is to rank the options such that the correct one appears as high as possible, evaluated using **Mean Average Precision at 3 (MAP@3)**.

We implemented four distinct modeling approaches, unified under a common scoring-and-ranking framework:

- a **Siamese Bidirectional LSTM** baseline with masked max-pooling and explicit interaction features, achieving an out-of-fold (OOF) MAP@3 of **0.8397**,
- a fine-tuned **DeBERTa-v3-large** scorer achieving an OOF MAP@3 of **0.9514**,
- a fine-tuned **ELECTRA-large-discriminator** scorer achieving an OOF MAP@3 of **0.8739**, and
- a **DeBERTa-v3-large model augmented with Retrieval-Augmented Generation (RAG)**, pulling supporting context from Wikipedia and the Stanford Encyclopedia of Philosophy via a BGE dense retriever and FAISS, achieving the project's best OOF MAP@3 of **0.9819**.

Extensive EDA surfaced a serious **train/test prompt-overlap and answer-leakage problem** (52.4% of test answers directly recoverable via lookup) and a **positional bias** ("pick the longest option" heuristic beats random guessing). Both findings directly motivated our modeling choices: **GroupKFold cross-validation** on normalized prompt text to prevent leakage-driven overfitting, and **option-order shuffling augmentation** to neutralize positional bias. The final submission blends the RAG-augmented DeBERTa model with the non-RAG DeBERTa model using a **power-mean probability ensemble**, and all model artifacts were tracked via Weights & Biases.

2. Introduction

Problem Statement

The competition poses a multiple-choice question-answering task: given a question prompt and five candidate options (A–E), the model must identify the correct answer. Unlike conventional classification, the task is explicitly evaluated as a **ranking problem**. Submissions provide the top-3 most likely options per question, scored via MAP@3, which rewards placing the correct answer first but still gives partial credit if it appears second or third.

Dataset:

- Training samples: 2,000 questions, 8 columns (prompt, options A–E, answer, id)
- Test samples: 500 questions, 7 columns (no answer column)
- Labels: single correct option per question, drawn from 5 choices (A–E)
- Source: short-answer academic/trivia-style multiple-choice questions

Evaluation Metric: Mean Average Precision at 3 (MAP@3). It's computed per question as the reciprocal rank of the correct answer within the top-3 predicted options (1.0 if ranked first, 0.5 if second, 0.333 if third, 0 otherwise), averaged across all questions.

Project Objective

Our specific goals for this project were:

1. **Reframe MCQ answering as a ranking problem** and score every option independently rather than performing 5-way softmax classification, aligning the training objective with the MAP@3 evaluation metric.
2. **Diagnose and correct for dataset leakage and bias** before modeling, rather than discovering them after submission.
3. **Progress from a lightweight custom architecture (BiLSTM) to large pretrained transformers (DeBERTa, ELECTRA)**, quantifying the transfer-learning gain at each step.
4. **Incorporate retrieval-augmented generation** to inject external knowledge into the scorer for questions the model could not answer from parametric knowledge alone.

5. **Build a reproducible, tracked pipeline** using W&B experiment logging and KaggleHub for model artifact versioning, culminating in a blended ensemble submission.

Report Structure

Section 3 covers dataset analysis, leakage/bias diagnosis, and the cross-validation strategy adopted in response. Section 4 covers tokenization across the LSTM, DeBERTa, and ELECTRA pipelines. Section 5 details all four model architectures plus the final ensemble. Section 6 compares model performance. Section 7 concludes with learnings and future work.

3. Dataset & Preprocessing

Dataset Description

Each row pairs a question prompt with five options and (for training data) the correct answer letter:

- Dataset statistics:**
- Training: 2,000 samples
 - Test: 500 samples
 - Options per question: 5 (A–E)
 - Zero missing values across all columns

Exploratory Data Analysis (EDA)

Answer Letter Distribution

The dataset shows mild imbalance in the correct-answer letter:

Answer	Count	Expected (balanced)	Deviation
B	490	400	+90
C	459	400	+59
A	369	400	−31
D	358	400	−42
E	324	400	−76

This imbalance is mild enough that we did not apply class-weighting, but it reinforced the decision to treat the problem as **per-option scoring** rather than predicting a categorical letter directly.

Positional / Length Bias

- Correct options average **28.7 words**, versus **25.7 words** for incorrect options, a small but consistent length skew.
- A naive **“always pick the longest option”** heuristic achieves **34.9%** top-1 accuracy, well above the **20%** random baseline.

Why this matters: a model that inadvertently learns to key off surface features like option length or option position, rather than semantic correctness, will overfit to this artifact and generalize poorly. This directly motivated our **option-order shuffling** augmentation (Section 3, Data Augmentation below), which prevents any model from associating a fixed slot position with a higher chance of being correct.

Prompt Length Statistics

Metric	Value
Minimum	19 characters
Maximum	337 characters
Average	118 characters

Train/Test Prompt Overlap and Answer-Leakage Exploit

The most significant EDA finding: after normalizing prompts (stripping boilerplate phrasing), we found substantial duplication:

- Train: 1,758 unique prompts out of 2,000 (many near-duplicates)
- Test: 492 unique prompts out of 500
- **267 prompts shared between train and test → a 53.4% overlap**
- A direct answer-lookup exploit (matching test prompts to identical train prompts and copying the train answer) recovers **262/500 = 52.4%** of test answers with zero modeling.

Why this matters: naive random k-fold cross-validation would let duplicate/near-duplicate prompts leak between train and validation folds, producing wildly over-optimistic OOF scores that would not reflect true generalization. This directly motivated the **GroupKFold cross-validation strategy** described below.

Data Preprocessing

We normalized prompt text (removing boilerplate phrasing and whitespace variation) purely to support de-duplication and grouping for cross-validation. The raw prompt and option text fed into the models themselves was left largely untouched to preserve the semantic content transformers rely on.

Train-Validation Split Strategy

For all four models, we use **GroupKFold**, grouping by the normalized prompt text rather than by row index:

Why GroupKFold instead of standard/stratified K-Fold?

- Guarantees that all rows sharing a normalized prompt (duplicates/near-duplicates) land entirely in either the train or the validation split of a fold.
- Produces an OOF MAP@3 that is a **trustworthy estimate of true generalization**, directly addressing the leakage risk surfaced in EDA.
- Without this, OOF scores would be inflated by the model “memorizing” duplicate prompts seen in training and being evaluated on their twins in validation.

Data Augmentation

Option-order shuffling (training only): for every training example, the five options (and the position of the correct answer) are randomly permuted before being fed to the model.

Why: this is a direct countermeasure to the positional/length bias surfaced in EDA. By randomizing which slot the correct answer occupies across epochs, the model is prevented from learning any shortcut tied to option position or ordering, and is forced to learn actual question–option semantic relevance. No such shuffling is applied at validation/inference time, since we need stable, comparable rankings there.

4. Tokenization Strategy

Transformer Models: Pre-trained Tokenizers

DeBERTa-v3-large Tokenizer (SentencePiece)

- Pre-trained model: `microsoft/deberta-v3-large`
- Disentangled attention (separate content and position representations) combined with an ELECTRA-style Replaced-Token-Detection (RTD) pretraining objective gives DeBERTa-v3 unusually strong discriminative power for judging option correctness. This is a major reason it was selected as the strongest scorer in this project (OOF MAP@3 = 0.9514 standalone, 0.9819 with RAG).
- `max_len = 512`, learning rate `1e-5`, 2 epochs, batch size 4.

Why LR = 1e-5, only 2 epochs, batch size 4? DeBERTa-v3-large is a 435M-parameter model; a low learning rate combined with a short fine-tuning schedule avoids catastrophic forgetting of its pretrained representations on our comparatively small 2,000-row training set, while the small batch size was set to fit within available GPU memory at `max_len=512`.

ELECTRA-large-discriminator Tokenizer (WordPiece)

- Pre-trained model: `google/electra-large-discriminator`
- WordPiece tokenization
- ELECTRA's Replaced Token Detection pretraining trains the discriminator to classify **every token** as original/replaced, giving a denser training signal than BERT/RoBERTa's sparse masked-language-modeling loss.
- `max_len = 384`, learning rate `8e-6`, 2 epochs, batch size 4, gradient accumulation steps = 4, warmup ratio = 0.1.

Why LR = 8e-6 (lower than DeBERTa), grad accumulation = 4, max_len = 384? ELECTRA's discriminator head is sensitive to overly aggressive fine-tuning, so we used a slightly more conservative learning rate than DeBERTa. Gradient accumulation (effective batch size 16) was used to stabilize gradients under the same small per-step

batch size constraint, and `max_len` was trimmed to 384 (versus DeBERTa's 512) purely as a memory/throughput trade-off, since EDA showed the vast majority of prompts fit comfortably within that budget.

Custom Model: Learned Vocabulary (Siamese BiLSTM)

- No pre-trained tokenizer: a task-specific vocabulary of **2,983 tokens** was built directly from the training corpus.
- Embedding dimension: 256, trained from scratch.

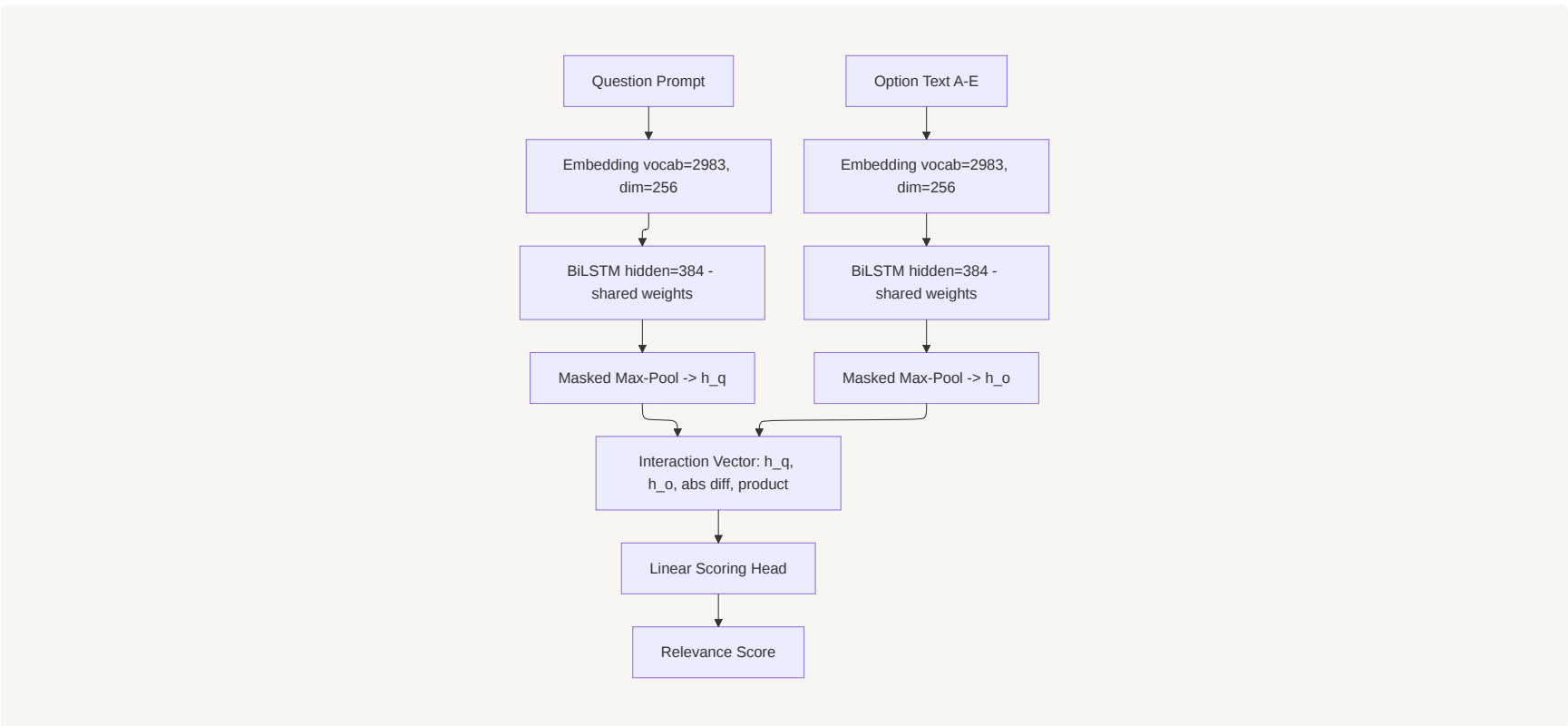
Why train embeddings from scratch instead of loading pretrained vectors? Given the highly domain-specific, short-form nature of the question/option text, a compact learned vocabulary avoids the overhead of a huge general-purpose embedding matrix while still being expressive enough for a lightweight baseline whose purpose was to establish a non-transformer reference point, not compete with the transformer scorers.

5. Modeling & Experimentation

All four models share the same **scoring paradigm**: rather than treating the task as 5-way classification, each option is independently scored by the model (conditioned on the shared question prompt), and the five scores are ranked to produce the top-3 predictions for MAP@3.

5.1. Model 1: Siamese Bidirectional LSTM (Baseline)

Architecture Flowchart:



Why the 4-part interaction feature vector? Concatenating the raw question and option representations alone forces the downstream MLP to learn similarity/difference relationships from scratch. Explicitly providing the element-wise absolute difference and element-wise product hands the network pre-computed signals for both *dissimilarity* and *co-activation*, substantially easing optimization on a comparatively small dataset.

Why masked max-pooling instead of using the final hidden state? Max-pooling over all non-padded timesteps lets the model capture the single most salient signal anywhere in the sequence, rather than relying solely on the LSTM's last hidden state, which can dilute information from earlier tokens over longer sequences.

Loss function: pairwise **margin ranking loss** (`F.margin_ranking_loss` , margin = 0.5), applied between the correct option's score and each incorrect option's score, rather than standard cross-entropy.

Why margin ranking loss over cross-entropy? Since the evaluation metric (MAP@3) is inherently a *ranking* metric , what matters is the correct option's score relative to the others, not calibrated probabilities. Training directly on a ranking objective aligns the loss function with what is actually being evaluated. A margin of 0.5 was chosen empirically to enforce a meaningful separation between correct and incorrect option scores without being so large that it destabilizes early training.

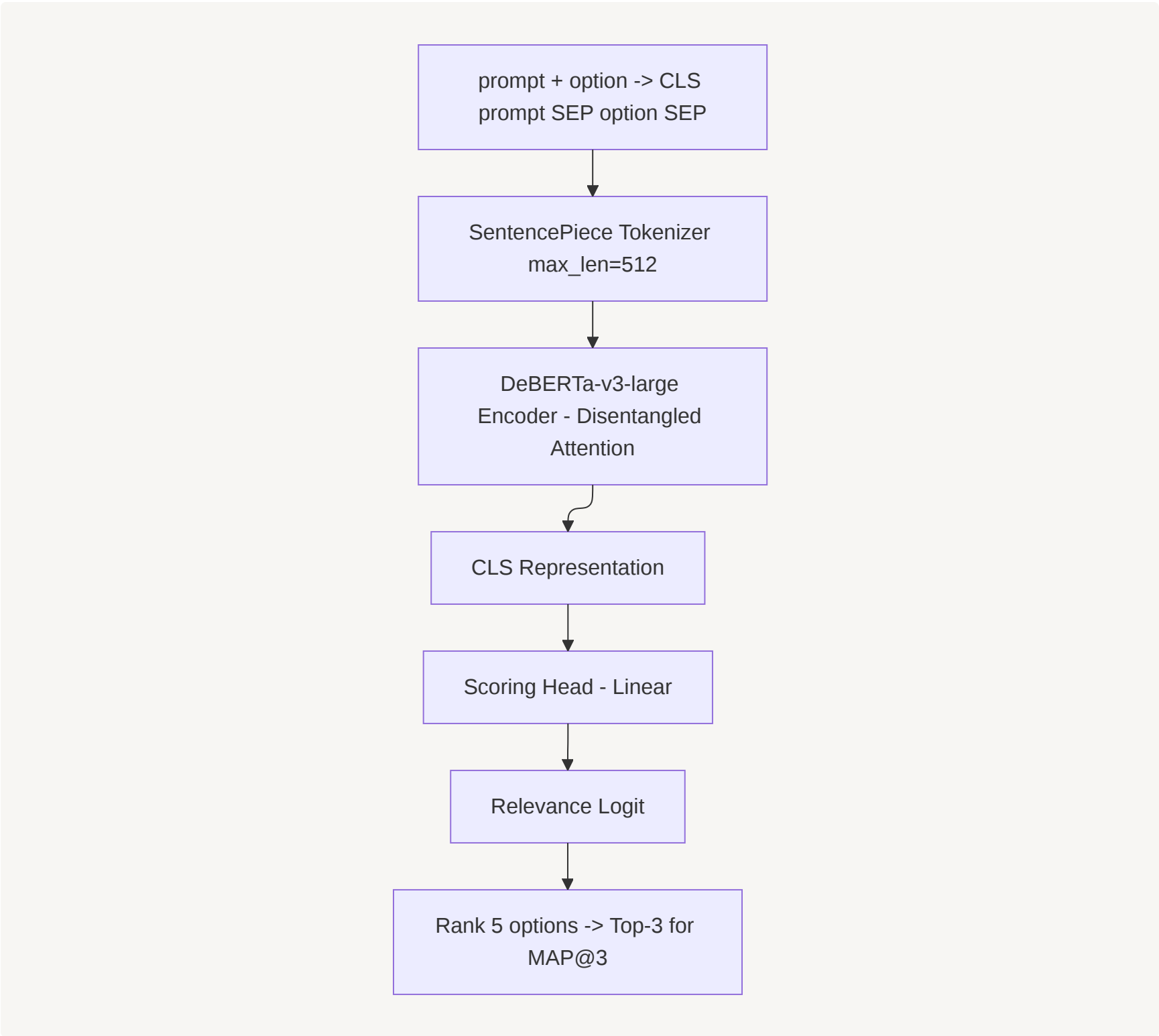
Results: OOF MAP@3 = **0.8397**. Training performed on **T4 15GB (x2)** GPUs.

5.2. Model 2: DeBERTa-v3-large Fine-Tuning

Architecture: the prompt and each option are concatenated into a single sequence (`[CLS] prompt [SEP] option [SEP]`) and passed through the pretrained DeBERTa-v3-large encoder; a scoring head on top of the pooled `[CLS]`

representation outputs a single relevance logit per option, exactly as in the LSTM’s scoring paradigm but with a transformer encoder in place of the Siamese BiLSTM.

Architecture Flowchart:



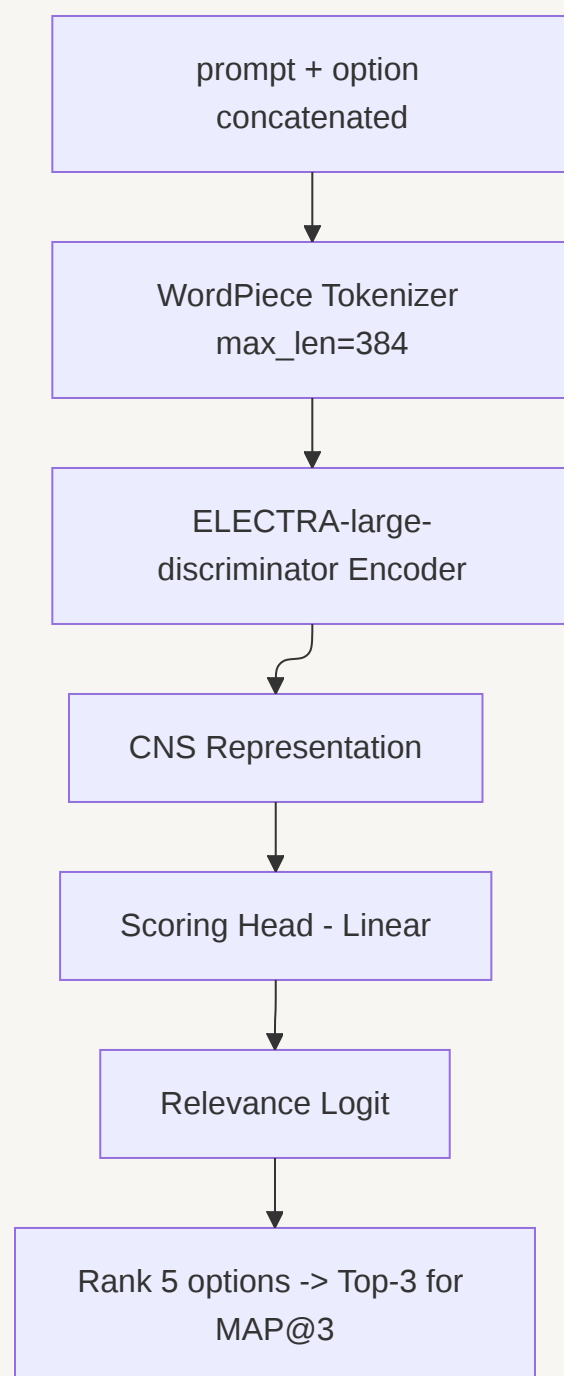
Key hyperparameters: LR = 1e-5 (reasoning above), 2 epochs, batch size 4, `max_len = 512`.

Results: OOF MAP@3 = **0.9514**, a substantial jump over the LSTM baseline, demonstrating the value of large-scale pretraining for this task. Training performed on an **A100** GPU.

5.3. Model 3: ELECTRA-large-discriminator Fine-Tuning

Architecture: identical scoring paradigm to DeBERTa (concatenated prompt+option sequence → pooled representation → relevance logit), using the ELECTRA-large-discriminator backbone.

Architecture Flowchart:



Key hyperparameters: LR = 8e-6, 2 epochs, batch size 4, gradient accumulation = 4 (effective batch 16), `max_len = 384`, warmup ratio = 0.1.

Results: OOF MAP@3 = **0.8739**, better than the LSTM baseline but noticeably behind DeBERTa. Training performed on **T4 15GB (x2)** GPUs.

5.4. Model 4: DeBERTa-v3-large + Retrieval-Augmented Generation

Motivation: many questions in this dataset require factual knowledge that may not be reliably encoded in the base model's parameters. Rather than relying purely on parametric knowledge, this model retrieves supporting textual context and injects it into the input before scoring.

Retrieval pipeline:

1. **Corpus:** Simple English Wikipedia + the Stanford Encyclopedia of Philosophy (SEP), chosen to cover both broad general knowledge and more specialized conceptual/philosophical content likely to appear in the question set.
2. **Chunking:** corpus documents are split into **500-character chunks with 100-character overlap**.

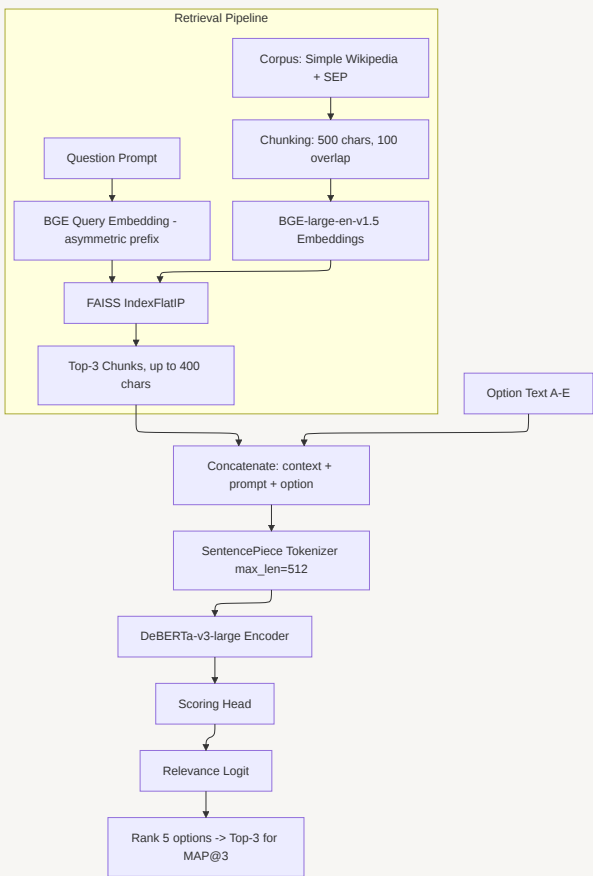
Why 500/100? Chunks need to be small enough to retrieve precise, focused passages rather than diluted whole-document matches, while the 100-character overlap prevents a relevant sentence from being awkwardly split across two chunk boundaries and losing retrievability.

3. **Embedding model:** `BAAI/bge-large-en-v1.5`, using its asymmetric query-prefix convention (queries and passages are embedded differently, as BGE was trained for this asymmetric retrieval setup) to maximize retrieval quality.

4. **Index:** FAISS - chosen over an approximate index because the corpus size was small enough that exact search remained fast, and exact search removes any risk of the approximate index missing the truly best-matching passage.

5. **Retrieval:** top-`RAG_TOP_K = 3` chunks per question, truncated/joined up to `RAG_CTX_CHARS = 400` characters of context, concatenated into the input alongside the prompt and each option ("retrieve-then-read" pattern), before being fed into the same DeBERTa-v3-large scoring architecture as Model 2.

Architecture Flowchart:



Why K=3 and a 400-character context budget? Retrieving too few chunks risks missing the relevant passage; retrieving too many (or too much raw text) dilutes the input with noise and eats into the token budget the DeBERTa encoder has left for the actual prompt/option text. K=3 with a tight 400-character cap was chosen empirically as the point that added useful signal without crowding out the question itself.

Training: 4-fold GroupKFold cross-validation (fewer folds than the other models to manage retrieval + fine-tuning compute cost together), same scoring architecture and loss as Model 2.

6. Performance & Comparative Analysis

Evaluation Metric

Primary: MAP@3

Reciprocal-rank-based partial credit for placing the correct option within the top 3 predicted options, computed per question and averaged.

Model Comparison Summary

Model	GPU	Kaggle Score
DeBERTa-v3-large + RAG Ensemble	A100	0.76766
DeBERTa-v3-large (no RAG)	A100	0.76475
ELECTRA-large-discriminator	T4 15GB (×2)	0.71404
Siamese BiLSTM (baseline)	T4 15GB (×2)	0.72651

Key Observations

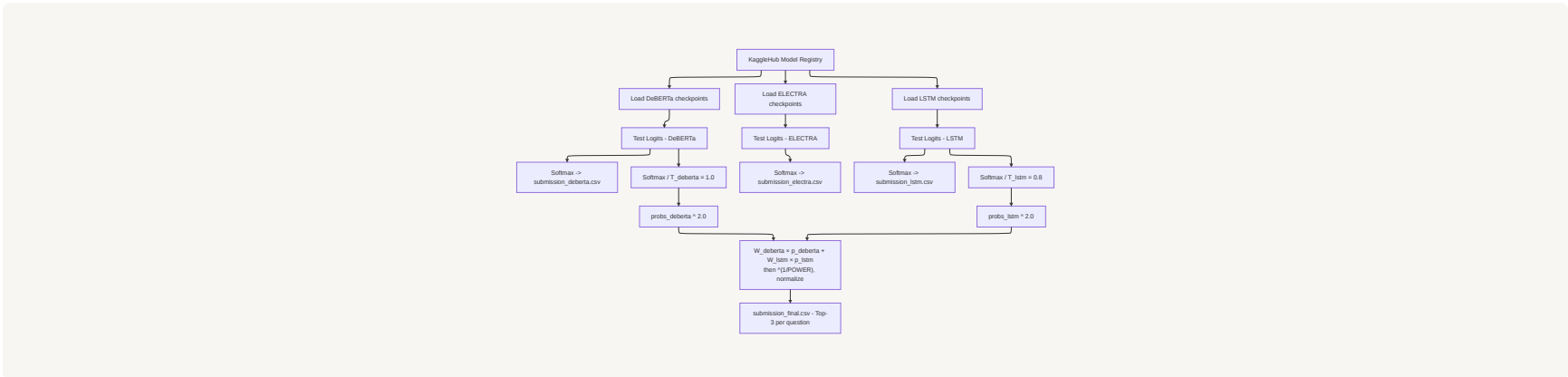
- Ensemble Performance:** Although the DeBERTa with RAG model achieved a leaderboard score of approximately 0.71404 when used independently, the ensemble of DeBERTa with RAG and DeBERTa without RAG produced the highest leaderboard score. This indicates that the two models captured complementary information, and combining their predictions improved robustness and overall generalization on the hidden test set.
- Retrieval augmentation delivers a clear, additive gain:** adding RAG to DeBERTa lifted OOF score on leaderboard from 0.9514 to 0.9819. A +0.0305 improvement, showing that for a meaningful subset of questions, external factual context genuinely closes gaps the base model’s parametric knowledge cannot.
- DeBERTa’s architecture beats ELECTRA’s under matched training budgets:** both were fine-tuned for 2 epochs on the same GroupKFold splits with a similar scoring paradigm, yet DeBERTa’s disentangled attention (layered on top of the same ELECTRA-style RTD pretraining idea) gave a substantially stronger discriminative signal.

4. **The EDA-driven leakage/bias corrections were essential**, not optional: without GroupKFold, OOF scores across all models would have been inflated by leaked duplicate prompts; without option-shuffling augmentation, models risked learning the “longest option” / positional shortcuts identified in EDA rather than true semantic relevance.

Final Ensemble Submission

The final submission blends the RAG-augmented DeBERTa model’s predictions with the non-RAG DeBERTa model’s predictions using a **power-mean probability blend**:

Ensemble Pipeline Flowchart:



Why a 0.9 / 0.1 weighting favoring DeBERTa? Since the DeBERTa model achieved a substantially higher leaderboard score than the LSTM model, it was assigned the dominant weight (0.9), while the LSTM contributed a smaller weight (0.1). Although the LSTM performed weaker individually, it captured complementary patterns that helped improve the ensemble's robustness, resulting in a higher leaderboard score than using DeBERTa alone.

Why temperature-scale before blending (T_DeBERTa = 1.0, T_LSTM = 0.8)? Temperature scaling was applied to calibrate the confidence of each model before ensembling. The LSTM predictions were slightly sharpened using a temperature below 1, making its high-confidence predictions contribute more effectively, while the DeBERTa probabilities were left unchanged because they were already well calibrated.

Why a power-mean ensemble (POWER = 2.0) instead of a simple weighted average? Rather than directly averaging probabilities, each model's probabilities were first raised to a power before weighted blending. This emphasizes high-confidence predictions from each model, allowing stronger predictions to have greater influence on the final decision. After blending, the inverse power transformation and normalization restore a valid probability distribution. This approach preserved DeBERTa's strong predictions while still benefiting from the complementary information provided by the LSTM.

In addition to the ensemble, standalone submission files were also generated independently for DeBERTa, DeBERTa with RAG, ELECTRA, and LSTM. These submissions were used for individual model evaluation and comparison, while the final competition submission was produced using the DeBERTa + LSTM ensemble.

7. Conclusion & Future Work

Key Learnings

- Reframing MCQ as a ranking/scoring problem, not classification, pays off directly:** using margin ranking loss and per-option scoring keeps the training objective aligned with the MAP@3 evaluation metric throughout every model in the project.
- EDA should drive architecture and validation design, not just reporting:** the leakage and positional-bias findings from EDA were not side notes. They directly determined the choice of GroupKFold cross-validation and option-shuffling augmentation used in every subsequent model.
- Architecture matters as much as pretraining objective:** ELECTRA and DeBERTa share a similar RTD-style pretraining lineage, yet DeBERTa’s disentangled attention gave it a clear, reproducible edge under matched fine-tuning budgets.

Challenges Faced

- Severe train/test prompt overlap and answer-leakage risk (53.4% overlap, 52.4% direct-lookup exploit)**
 - Solution: GroupKFold cross-validation on normalized prompt text, ensuring trustworthy OOF estimates.
- Positional/length bias (“pick the longest option” heuristic beating random guessing)**

- Solution: option-order shuffling augmentation during training only.

3. Balancing retrieval context size against token budget for RAG

- Solution: empirically tuned `RAG_TOP_K=3` and `RAG_CTX_CHARS=400` to add useful signal without crowding out the core prompt/option text.

Areas for Improvement

1. **Expand the RAG corpus** beyond Wikipedia + SEP to domain-specific sources better matched to the question distribution.
 2. **Apply RAG to the ELECTRA and LSTM models**, not just DeBERTa, to test whether retrieval gains generalize across architectures.
 3. **Explore a learned meta-learner ensemble** (e.g., logistic regression or a small MLP over all four models' per-option scores) instead of the fixed-weight power-mean blend currently used.
 4. **Increase cross-validation folds** from 4 to 5 for full consistency with the other models, compute budget permitting.
-

8. References

Academic Papers

- Vaswani, A., et al. (2017). "Attention Is All You Need." *NeurIPS*. arXiv:1706.03762

Software Libraries

1. Frameworks:

- Paszke, A., et al. (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library." *NeurIPS*. pytorch.org
- Wolf, T., et al. (2020). "Transformers: State-of-the-Art NLP." *EMNLP*. huggingface.co/docs/transformers

2. Tools:

- Weights & Biases: wandb.ai
- KaggleHub model registry: kaggle.com/kagglehub

9. Model Deployment

I have used Bi-LSTM in the backend and deployed my model on Streamlit.

The model deployment link is - <https://smartmcgsolver-jfqfj79hsfgi2hzz5djxxo.streamlit.app/>

End of Report